

O que é React?

React (ou React.js) é uma **biblioteca** JavaScript de código aberto mantida pelo Meta (Facebook) para construir interfaces de usuário (UI), especialmente para aplicações web de página única (SPA).

Principais características:

- **Componentes:** a UI é dividida em peças reutilizáveis e independentes (componentes), cada um com sua própria lógica e aparência.
- **Declarativo:** você descreve *como* a UI deve parecer para cada estado, e o React cuida de atualizar e renderizar os componentes corretamente quando os dados mudam.
- **Virtual DOM:** em vez de manipular o DOM real diretamente, o React mantém uma representação virtual em memória. Quando o estado muda, ele calcula a diferença e aplica apenas as atualizações necessárias no DOM real, tornando as atualizações eficientes.
- **Unidirecional:** os dados fluem de cima para baixo (do componente pai para o filho) via **props**.
- **Estado (state):** componentes podem ter estado interno que, quando alterado, dispara uma re-renderização.

Exemplo mínimo:

```
function Saudacao({ nome }) {  
  return <h1>Olá, {nome}</h1>;  
}
```

Para que serve?

- Aplicações web modernas e dinâmicas (dashboards, redes sociais, e-commerce).
- Aplicações mobile (com React Native).
- Interfaces complexas que exigem alta performance e manutenibilidade.

React não é um framework completo (como Angular ou Vue), mas sim uma biblioteca focada na camada de **VIEW**.

Para funcionalidades como roteamento, gerenciamento de estado global ou chamadas HTTP, você geralmente combina com outras bibliotecas (React Router, Redux, Axios, etc.).

CREATE REACT ***

- **Create React App** → npx create-react-app (CRA)
- **Vite** → npm create vite@latest (com template React)
- **Next.js** → npx create-next-app

A diferença principal está no **propósito**, **performance** e **recursos** de cada ferramenta.

1. Create React App (CRA)

- **Foco:** Aplicação React **client-side** (SPA) pura.
- **Prós:**
 - Configuração zero, pronto para uso.
 - Grande ecossistema e documentação.
- **Contras:**
 - **Lento** para iniciar o servidor de desenvolvimento (especialmente em projetos grandes).
 - Build mais demorada.
 - Manutenção desacelerada (Meta praticamente abandonou; não recebe atualizações significativas desde 2022).
 - Projetos legados.

2. Vite (com React)

- **Foco:** Aplicação React **client-side** (SPA), mas extremamente rápida.
- **Prós:**
 - Build muito mais rápida que CRA.
 - Suporte nativo a TypeScript, JSX, CSS Modules, etc.
 - Leve e flexível.
- **Contras:**
 - você precisa configurar algumas coisas manualmente, como variáveis de ambiente com prefixo VITE_).
 - Ecossistema de plugins ainda crescendo (mas já muito maduro).

3. Next.js (create-next-app)

- **Tipo:** Framework React full-stack.
- **Foco:** Aplicações React com **renderização no servidor** (SSR), geração estática (SSG), API routes, etc.
- **Prós:**

- Melhor SEO e performance inicial.
- Roteamento baseado em arquivos (file-system routing).
- API Routes (backend integrado).
- **Contras:**
 - Mais complexo (curva de aprendizado maior).
 - Pode ser "overkill" (exagero) para SPAs simples.
 - Consome mais recursos no servidor.
- **Recomendado para:** Aplicações que precisam de SEO, performance de carregamento, ou que combinam frontend e backend.

Qual escolher?

- Projeto novo SPA simples?
 - Vite (rápido, moderno, recomendado).
- Projeto que precisa de SEO ou renderização no servidor?
 - Next.js.
- Projeto legado ou time já acostumado com CRA?
 - Pode continuar com CRA, mas considere migrar para Vite.

Observação: A própria documentação oficial do React recomenda o Next.js ou ferramentas como Vite para novos projetos.

O que são Hooks no React?

Hooks são funções especiais introduzidas no React 16.8 (2019) que permitem usar **estado** e outros recursos do React **em componentes funcionais**, sem precisar escrever uma classe.

Antes dos hooks, apenas componentes de classe podiam ter estado e ciclo de vida. Com os hooks, componentes funcionais se tornaram tão poderosos quanto classes, mas com código mais limpo e reutilizável.

Por que os hooks foram criados?

- **Reutilizar lógica entre componentes** sem usar padrões complexos.
- **Organizar melhor o código**, lógica relacionada fica junta, não espalhada em métodos de ciclo de vida.
- **Evitar o this confuso** das classes JavaScript.
- **Componentes menores e mais focados.**

Regras dos Hooks

1. **Chame hooks apenas no nível superior** do componente (não dentro de loops, condicionais ou funções aninhadas).
2. **Chame hooks apenas em componentes funcionais do React** ou em outros hooks customizados.
3. O nome do hook deve começar com use (ex: useState, useEffect).

useState: Estado local

Gerencia valores que mudam ao longo do tempo e causam re-renderização.

```
import { useState } from 'react';

function Contador() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
    </div>
  );
}
```

useEffect: Efeitos colaterais

Executa código após a renderização (requisições HTTP, manipulação de DOM, timers, subscriptions).

```
import { useState, useEffect } from 'react';

useEffect(() => {
  const response = await fetch( 'https://jsonplaceholder.typicode.com/users',
    { signal: abortController.signal } );

}, [])
```

Existem vários hooks no React, os hooks transformaram a forma como escrevemos React, componentes mais simples, lógica reutilizável e sem classes. Dominar useState e useEffect já cobre a maioria dos casos do dia a dia.

UseState

1. Pré-requisitos

- Node.js **18.17** ou superior instalado
- npm, yarn ou pnpm
 - node -v
 - Npm -v

2. Criar o projeto Next.js

Abra o terminal e execute:

- `npx create-next-app@latest contador-app`

Durante a instalação, você pode aceitar as opções padrão (TypeScript, ESLint, App Router, etc.).

Para nossa aula vamos usar **JavaScript** e **App Router** (padrão).

```
PS F:\Aula_2026-02\4ºADS_Script> npx create-next-app@latest contador-app
✓ Would you like to use the recommended Next.js defaults? » No, customize settings
✓ Would you like to use TypeScript? ... No / Yes
✓ Which linter would you like to use? » None
✓ Would you like to use React Compiler? ... No / Yes
✓ Would you like to use Tailwind CSS? ... No / Yes
✓ Would you like your code inside a `src/` directory? ... No / Yes
✓ Would you like to use App Router? (recommended) ... No / Yes
✓ Would you like to customize the import alias (`@/*` by default)? ... No / Yes
✓ What import alias would you like configured? ... @/*
? Would you like to include AGENTS.md to guide coding agents to write up-to-date Next.js code? » No / Yes
```

Após a criação, entre na pasta:

- `cd contador-app`
- `npm run dev`
- Acesse `http://localhost:3000` [você verá a página inicial do Next.js].

3. Estrutura do projeto (App Router)

Dentro da pasta `app/`, temos:

```
app/
├─ layout.js    # Layout principal (HTML, body, etc.)
├─ page.js      # Página inicial (rota /)
├─ globals.css  # Estilos globais
```

JSX

JSX (JavaScript XML) é uma extensão de sintaxe para JavaScript que se parece com HTML, mas é usada dentro de código JavaScript. Ela foi criada pelo time do React para descrever como a interface do usuário deve parecer.

Exemplo básico

```
const elemento = <h1>Olá, mundo!</h1>;
```

Isso não é HTML, nem string, é **JSX**. Ele é transformado (compilado) em chamadas de `React.createElement` antes de ser executado no navegador.

O que acontece por baixo dos panos

O código JSX acima é convertido para:

```
const elemento = React.createElement('h1', null, 'Olá, mundo!');
```

Que por sua vez produz um objeto JavaScript (um "elemento React"):

```
{
  type: 'h1',
  props: { children: 'Olá, mundo!' }
}
```

Características principais

Expressões JavaScript dentro de chaves {}

```
const nome = 'João';
const elemento = <h1>Olá, { nome }!</h1>; // "Olá, João!"
```

Atributos em camelCase (diferente do HTML)

```
// HTML: class, onclick, tabindex
// JSX: className, onClick, tabIndex

<button className="btn" onClick={handleClick}>
  Clique aqui
</button>
```

Componentes com letra maiúscula

```
// Elemento HTML nativo (letra minúscula)
<div />
```

```
// Componente React (letra maiúscula)
<MeuComponente />
```

Retorno de um único elemento pai (ou fragment)

```
// Errado (dois elementos irmãos sem pai)
return (
  <h1>Título</h1>
  <p>Parágrafo</p>
);
```

```
// Correto (envolto em um fragment)
return (
  <>
    <h1>Título</h1>
    <p>Parágrafo</p>
  </>
);
```

Observação: JSX não é obrigatório, você pode escrever React puro com `React.createElement`, mas JSX torna o código muito mais legível.

Resumo

	HTML	JSX
Atributo class	<code>class="..."</code>	<code>className="..."</code>
Atributo for	<code>for="..."</code>	<code>htmlFor="..."</code>
Estilo inline	<code>style="color: red"</code>	<code>style={{ color: 'red' }}</code>
Comentários	<code><!-- ... --></code>	<code>{/* ... */}</code>
Expressões	Não suporta	Suporta com <code>{}</code>

JSX não é HTML, nem um template string, mas sim JavaScript com uma sintaxe declarativa que descreve a UI.

Criar o componente Contador

Crie uma pasta **components/** dentro da raiz do projeto.

Crie o arquivo `app/components/Contador.jsx`:

```
'use client'; // Necessário no App Router para usar hooks

import { useState } from 'react';

export default function Contador() {

  // useState retorna um array: [valorAtual, funçãoParaAtualizar]
  const [contador, setContador] = useState(0);

  const incrementar = () => {
    setContador(contador + 1);
  };

  const decrementar = () => {
    setContador(contador - 1);
  };

  return (
    <div style={{ textAlign: 'center', marginTop: '2rem' }}>
      <h1>Contador: {contador}</h1>
      <button onClick={incrementar} style={botaoStyle}>
        +1
      </button>
      <button onClick={decrementar} style={botaoStyle}>
        -1
      </button>
    </div>
  );
}

// Estilo simples inline (apenas para exemplo)
const botaoStyle = {
  margin: '0 10px',
  padding: '10px 20px',
  fontSize: '1.2rem',
  cursor: 'pointer',
};
```

Usar o componente na página principal

Substitua o conteúdo de `app/page.js` por:

```
import Contador from './components/Contador';
```

```
export default function Home() {  
  return (  
    <main>  
      <Contador />  
    </main>  
  );  
}
```

Nota: O componente Contador tem a diretiva '**use client**' porque usa hooks (useState). Componentes sem essa diretiva são renderizados no servidor por padrão no App Router.

Conceito-chave: useState

```
const [estado, setEstado] = useState(valorInicial);
```

- Estado: variável que armazena o valor atual.
- setEstado: função que atualiza o estado e dispara uma nova renderização do componente.
- valorInicial: valor inicial (pode ser número, string, objeto, array, etc.).

Boas práticas:

- Nunca modifique o estado diretamente (contador = 5). Sempre use a função setContador.
- Se o novo estado depender do anterior, use a forma funcional:

```
setContador(prev => prev + 1);
```

Exercício: Contador Duplo com useState

Objetivo: Criar um componente React que gerencie dois contadores independentes usando useState. Cada contador deve ter seus próprios botões de incrementar e decrementar, e um botão para resetar individualmente.

Enunciado

- Crie um componente funcional chamado **ContadorDuplo**.
- Utilize dois estados com useState:
 - contadorA (inicial 0)
 - contadorB (inicial 10)
- Para cada contador, renderize:
 - O valor atual
 - Um botão +1
 - Um botão -1
 - Um botão Resetar (volta ao valor inicial)
- Os contadores devem funcionar de forma totalmente independente.
- Adicione um botão global **Zerar Tudo** que reseta ambos os contadores para 0.

Código comentado

```
import { useState } from 'react';

function ContadorDuplo() {

  // ===== ESTADOS INDEPENDENTES =====
  const [contadorA, setContadorA] = useState(0); // primeiro contador
  const [contadorB, setContadorB] = useState(10); // segundo contador (inicia em 10)

  // ===== FUNÇÕES PARA CONTADOR A =====
  const incrementarA = () => setContadorA(prev => prev + 1);
  const decrementarA = () => setContadorA(prev => prev - 1);
  const resetarA = () => setContadorA(0);

  // ===== FUNÇÕES PARA CONTADOR B =====
  const incrementarB = () => setContadorB(prev => prev + 1);
  const decrementarB = () => setContadorB(prev => prev - 1);
  const resetarB = () => setContadorB(10); // volta ao valor inicial

  // ===== FUNÇÃO GLOBAL =====
  const zerarTudo = () => {
    setContadorA(0);
    setContadorB(0);
  };

  // ===== RENDERIZAÇÃO =====
  return (
```

```

<div style={{ textAlign: 'center', fontFamily: 'Arial' }}>
  <h1>Contador Duplo</h1>

  {/ * ---- CONTADOR A ---- */}
  <div style={{ margin: '20px', padding: '15px', border: '1px solid #ccc' }}>
    <h2>Contador A</h2>
    <p style={{ fontSize: '2rem' }}>{contadorA}</p>
    <button onClick={incrementarA}>+1</button>
    <button onClick={decrementarA}>-1</button>
    <button onClick={resetarA}>Resetar</button>
  </div>

  {/ * ---- CONTADOR B ---- */}
  <div style={{ margin: '20px', padding: '15px', border: '1px solid #ccc' }}>
    <h2>Contador B</h2>
    <p style={{ fontSize: '2rem' }}>{contadorB}</p>
    <button onClick={incrementarB}>+1</button>
    <button onClick={decrementarB}>-1</button>
    <button onClick={resetarB}>Resetar</button>
  </div>

  {/ * ---- AÇÃO GLOBAL ---- */}
  <button
    onClick={zerarTudo}
    style={{ marginTop: '20px', padding: '10px 20px', background: '#ff4444', color: '#fff',
      border: 'none', cursor: 'pointer' }}
  >
    Zerar Tudo
  </button>
</div>
);
}

export default ContadorDuplo;

```

Links:

<https://react.dev/>